

1. Why Learning Rates Cannot be Too Big

Gradient descent has a speed limit, and the data sets it. The largest singular value of the problem decides how big a step you can take before the iterates blow up. One learning rate has to serve every direction at once, though, and the directions with small singular values want a much bigger step than the large ones can tolerate. By the end of this problem you will be able to write down that limit, choose the step size that balances the two, and see what the ratio of the largest to the smallest singular value costs you.

To understand the role of the learning rate, it is useful to understand it in the context of the simplest possible problem first.

Suppose that we want to solve the scalar equation

$$\sigma w = y \quad (1)$$

where we know that $\sigma > 0$. We proceed with an initial condition $w_0 = 0$ by using gradient descent to minimize the squared loss

$$L(w) = (y - \sigma w)^2 \quad (2)$$

which has a derivative with respect to the parameter w of $-2\sigma(y - \sigma w)$.

Gradient descent with a learning rate of η follows the recurrence-relation or discrete-time state evolution of:

$$\begin{aligned} w_{t+1} &= w_t + 2\eta\sigma(y - \sigma w_t) \\ &= (1 - 2\eta\sigma^2)w_t + 2\eta\sigma y. \end{aligned} \quad (3)$$

- (a) **For what values of learning rate $\eta > 0$ is the recurrence (3) stable?**

(HINT: Remember the role of the unit circle in determining the stability or instability of such recurrences. If you keep taking higher and higher positive integer powers of a number, what does that number have to be like for this to converge?)

- (b) The previous part gives you an upper bound for the learning rate η that depends on σ beyond which we cannot safely go. **If η is below that upper bound, how fast does w_t converge to its final solution $w^* = \frac{y}{\sigma}$? i.e. if we wanted to get within a factor $(1 - \epsilon)$ of w^* , how many iterations t would we need?**

(HINT: The absolute value of the error of current w to the optimality might help.)

- (c) Suppose that we now have a vector problem where we have two parameters $w[1], w[2]$. One with a large σ_ℓ and the other with a tiny σ_s . i.e. $\sigma_\ell \gg \sigma_s$ and we have the vector equation we want to solve:

$$\begin{bmatrix} \sigma_\ell & 0 \\ 0 & \sigma_s \end{bmatrix} \begin{bmatrix} w[1] \\ w[2] \end{bmatrix} = \begin{bmatrix} y[1] \\ y[2] \end{bmatrix}. \quad (4)$$

We use gradient descent with a single learning rate η to solve this problem starting from an initial condition of $\mathbf{w} = \mathbf{0}$.

For what learning rates $\eta > 0$ will we converge? Which of the two σ_i is limiting our learning rate?

- (d) **For the previous problem, depending on $\eta, \sigma_\ell, \sigma_s$, which of the two dimensions is converging faster and which is converging slower?**
- (e) The speed of convergence overall will be dominated by the slower of the two. **For what value of η will we get the fastest overall convergence to the solution?**
- (f) Comment on what would happen if we had more parallel problems with σ_i that all were in between σ_ℓ and σ_s ? **Would they influence the choice of possible learning rates or the learning rate with the fastest convergence?**
- (g) Using what you know about the SVD, **how is the simple scalar and parallel scalar problem analysis above relevant to solving general least-squares problems of the form $X\mathbf{w} \approx \mathbf{y}$ using gradient descent?**

2. Where Gradient Descent Cannot Go

When a model has more parameters than training points, infinitely many weight vectors fit the data perfectly, and the loss is the same at all of them. Gradient descent still ends up at one of them. This problem works out which one. Each step is a combination of the training inputs, so the iterates never leave $\mathbf{w}_0$ plus the row space of X , and whatever $\mathbf{w}_0$ had in the orthogonal directions stays there. The limit is the interpolating solution closest to $\mathbf{w}_0$. From the origin that is the minimum-norm solution. From anywhere else it is not.

Let $X \in \mathbb{R}^{n \times d}$ with $n < d$ and full row rank n , so $X\mathbf{w} = \mathbf{y}$ has infinitely many solutions. Run gradient descent on $L(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|_2^2$ from a starting point $\mathbf{w}_0$:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + 2\eta X^\top \mathbf{r}_t, \quad \mathbf{r}_t := \mathbf{y} - X\mathbf{w}_t. \quad (5)$$

Recall that $\mathbb{R}^d$ is the orthogonal direct sum of the row space $\text{range}(X^\top)$ and the null space $\text{null}(X)$, so every $\mathbf{w}$ splits uniquely as $\mathbf{w} = \mathbf{w}^\parallel + \mathbf{w}^\perp$ with $\mathbf{w}^\parallel \in \text{range}(X^\top)$ and $\mathbf{w}^\perp \in \text{null}(X)$.

- (a) **Show that every gradient descent step lies in $\text{range}(X^\top)$, so that $\mathbf{w}_t \in \mathbf{w}_0 + \text{range}(X^\top)$ for all t . Conclude that the null-space component never moves: $\mathbf{w}_t^\perp = \mathbf{w}_0^\perp$ for all t .**
- (b) Let $X = U\Sigma V^\top$ be the full SVD, with $\sigma_1 \geq \dots \geq \sigma_n > 0$ and $\mathbf{v}_1, \dots, \mathbf{v}_d$ the columns of V . **Which $\mathbf{v}_i$ span $\text{null}(X)$? How many of the d coordinates $\mathbf{v}_i^\top \mathbf{w}_t$ can gradient descent change, and how many are fixed by $\mathbf{w}_0$?**
- (c) Suppose gradient descent converges to some $\mathbf{w}_\infty$ with $X\mathbf{w}_\infty = \mathbf{y}$. (You may assume η is small enough for this; the learning-rate problem says how small.) **Show that $\mathbf{w}_\infty$ is the unique solution of**

$$\min_{\mathbf{w}} \|\mathbf{w} - \mathbf{w}_0\|_2 \quad \text{subject to} \quad X\mathbf{w} = \mathbf{y}. \quad (6)$$

(Hint: for any other interpolating $\mathbf{w}$, split $\mathbf{w} - \mathbf{w}_0 = (\mathbf{w} - \mathbf{w}_\infty) + (\mathbf{w}_\infty - \mathbf{w}_0)$. Which subspace does each term lie in?)

- (d) **Specialize part (c) to $\mathbf{w}_0 = \mathbf{0}$: write $\mathbf{w}_\infty$ in closed form and name the estimator from the earlier SVD problem that it equals. Then say what changes if $\mathbf{w}_0 \neq \mathbf{0}$. Is the limit still the minimum-norm solution?**

3. An Alternate MAP Interpretation of Ridge Regression

Ridge regression has two formulas that look nothing alike. One inverts a $d \times d$ matrix and the other inverts an $n \times n$ matrix, so when you have far more features than training points, only one of them is practical to compute. This problem shows they give the same estimator. The proof does not push matrices around. It reads ridge as a MAP estimate under a Gaussian prior, and the second formula falls out of the usual formula for conditioning one jointly Gaussian vector on another.

Consider the Ridge Regression estimator,

$$\underset{\mathbf{w}}{\operatorname{argmin}} \|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda\|\mathbf{w}\|^2$$

We know this is solved by

$$\hat{\mathbf{w}} = (X^T X + \lambda I)^{-1} X^T \mathbf{y} \quad (7)$$

An alternate form of the Ridge Regression solution (often called the Kernel Ridge form) is given by

$$\hat{\mathbf{w}} = X^T (X X^T + \lambda I)^{-1} \mathbf{y}. \quad (8)$$

We know that Ridge Regression can be viewed as finding the MAP estimate when we apply a prior on the (now viewed as random parameters) $\mathbf{W}$. In particular, we can think of the prior for $\mathbf{W}$ as being $\mathcal{N}(\mathbf{0}, I)$ and view the random Y as being generated using $Y = \mathbf{x}^T \mathbf{W} + \sqrt{\lambda}N$ where the noise N is distributed iid (across training samples) as $\mathcal{N}(0, 1)$. At the vector level, we have $\mathbf{Y} = X\mathbf{W} + \sqrt{\lambda}\mathbf{N}$, and then we know that when we try to maximize the log likelihood we end up minimizing

$$\underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{\lambda} \|X\mathbf{w} - \mathbf{y}\|_2^2 + \|\mathbf{w}\|^2 = \underset{\mathbf{w}}{\operatorname{argmin}} \|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda\|\mathbf{w}\|^2.$$

The underlying probability space is that defined by the d iid standard normals that define the $\mathbf{W}$ and the n iid standard normals that give the n different N_i on the training points. Note that the X matrix, whose rows consist of the n different inputs for the n different training points, is not random.

Based on what we know about joint normality, it is clear that the random Gaussian vectors $\mathbf{W}$ and $\mathbf{Y}$ are jointly normal. **Use the following facts to show that the two forms of solution are identical.**

- (7) is the MAP estimate for $\mathbf{W}$ given an observation $\mathbf{Y} = \mathbf{y}$ (we showed this in HW0, and in discussion section)
- For jointly normal random variables, when you condition one set of variables on the values for the others, the resulting conditional distribution is still normal.
- A normal random variable has its density maximized at its mean.
- For jointly normal random vectors that are zero mean, the formula for conditional expectation is

$$E[\mathbf{W}|\mathbf{Y} = \mathbf{y}] = \Sigma_{WY} \Sigma_Y^{-1} \mathbf{y} \quad (9)$$

where the Σ_{YY} is the covariance $E[\mathbf{Y}\mathbf{Y}^T]$ of $\mathbf{Y}$ and $\Sigma_{WY} = E[\mathbf{W}\mathbf{Y}^T]$ is the appropriate cross-covariance of $\mathbf{W}$ and $\mathbf{Y}$.

4. Early Stopping is Ridge Regression in Disguise

In HW0 you saw several ways to think about ridge regression. Here is one more. Run ordinary gradient descent on the least-squares loss, with no penalty term, and stop before it has converged. We will show that after t steps the weights look like the ridge solution with λ roughly $\frac{1}{2\eta t}$, so stopping earlier acts like using a larger λ . We will also see when this stops being true.

Let $X \in \mathbb{R}^{n \times d}$ have full SVD $X = U\Sigma V^\top$ with singular values $\sigma_1 \geq \dots \geq \sigma_r > 0$, where r is the rank of X . Gradient descent starts at $\mathbf{w}_0 = \mathbf{0}$ and takes steps

$$\mathbf{w}_{t+1} = \mathbf{w}_t + 2\eta X^\top (\mathbf{y} - X\mathbf{w}_t) \quad (10)$$

on the loss $L(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|_2^2$. Write $\tilde{\mathbf{y}} = U^\top \mathbf{y}$ and $\tilde{\mathbf{w}}_t = V^\top \mathbf{w}_t$ for the coordinates of $\mathbf{y}$ and $\mathbf{w}_t$ in the singular vector bases.

Recall from HW0 that the ridge solution $\mathbf{w}_{\text{ridge}}(\lambda) = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$ has coordinates $\sigma_i \tilde{y}_i / (\sigma_i^2 + \lambda)$ in the V basis. We will write this as

$$[V^\top \mathbf{w}_{\text{ridge}}(\lambda)]_i = s_i^{\text{ridge}}(\lambda) \cdot \frac{\tilde{y}_i}{\sigma_i} \quad \text{with} \quad s_i^{\text{ridge}}(\lambda) = \frac{\sigma_i^2}{\sigma_i^2 + \lambda} \quad \text{for } i \leq r, \quad \text{and } 0 \text{ for } i > r. \quad (11)$$

The number $\tilde{y}_i / \sigma_i$ is the least-squares answer along $\mathbf{v}_i$; it is the i -th coordinate of the minimum-norm solution from Problem 2. Ridge multiplies it by $s_i^{\text{ridge}}(\lambda)$, which lies between 0 and 1. We will call a multiplier like this a *shrinkage factor*. It is close to 1 when σ_i^2 is much larger than λ , so those directions are kept, and close to 0 when σ_i^2 is much smaller than λ , so those directions are mostly thrown away.

(a) Show that in the V basis the update (10) becomes d separate scalar recurrences,

$$[\tilde{\mathbf{w}}_{t+1}]_i = (1 - 2\eta\sigma_i^2)[\tilde{\mathbf{w}}_t]_i + 2\eta\sigma_i\tilde{y}_i \quad \text{for } i \leq r, \quad [\tilde{\mathbf{w}}_{t+1}]_i = [\tilde{\mathbf{w}}_t]_i \quad \text{for } i > r. \quad (12)$$

This is the same change of coordinates you used in Problem 1, applied to the whole matrix at once.

(b) Assume that

$$0 < 2\eta\sigma_i^2 < 1 \quad \text{for every } i \leq r. \quad (13)$$

This is the condition from Problem 1 under which each recurrence in (12) converges.

Solve (12) starting from $\mathbf{w}_0 = \mathbf{0}$ and show that

$$[\tilde{\mathbf{w}}_t]_i = s_i^{\text{GD}}(t) \cdot \frac{\tilde{y}_i}{\sigma_i} \quad \text{with} \quad s_i^{\text{GD}}(t) = 1 - (1 - 2\eta\sigma_i^2)^t \quad \text{for } i \leq r, \quad \text{and } 0 \text{ for } i > r. \quad (14)$$

Then show that $s_i^{\text{GD}}(t)$ lies between 0 and 1, increases with t , and tends to 1 as $t \rightarrow \infty$. So gradient descent also multiplies the least-squares answer along $\mathbf{v}_i$ by a shrinkage factor, and this factor grows toward 1 as training continues.

(c) Now compare the two shrinkage factors, $s_i^{\text{ridge}}(\lambda)$ from (11) and $s_i^{\text{GD}}(t)$ from (14).

Show that $s_i^{\text{GD}}(t) \geq 1 - e^{-2\eta t \sigma_i^2}$. Then set $\lambda = \frac{1}{2\eta t}$ and compute both factors for a direction with $\sigma_i^2 = \lambda$. (Hint: $1 - x \leq e^{-x}$, and $(1 - 1/t)^t \rightarrow e^{-1}$.) Running gradient descent for t steps acts like ridge regression with which value of λ ? What happens to this value as t grows?

- (d) Now suppose the condition (13) fails for some direction i , that is, $2\eta\sigma_i^2 > 1$.

Show that $s_i^{\text{GD}}(t)$ is then larger than 1 for every odd t . A shrinkage factor larger than 1 means the estimate along $\mathbf{v}_i$ is larger in magnitude than the least-squares answer $\tilde{y}_i/\sigma_i$. **Can this happen with ridge regression for some choice of $\lambda > 0$? Show why or why not.**

5. Coding: ReLU Networks and Their Elbows

A ReLU hidden layer is a pile of elbows. Each unit adds one kink to the function, and where that kink sits is a parameter like any other. When you train every layer, the elbows slide toward the places where the target function bends. When you train only the output layer, they stay wherever the initialization dropped them. Watch for that difference in this notebook, because it is what feature learning looks like in this setting.

Work through `q_relu_vis.ipynb`. It fits a one-hidden-layer ReLU network to a piecewise-linear target at several hidden widths, under three regimes: SGD on the output layer only, SGD on all layers, and a closed-form ridge regression on the hidden-layer features. Training takes a few minutes, so start it running and read ahead while you wait. Submit a `.pdf` export of the completed notebook along with your write-up, and answer the questions below.

- After enough training, the test error sits *below* the train error, the opposite of what we usually expect. **What causes this?**
- How does the hidden layer width change the learned function and the test error? If you could place the elbows by hand, where would you put them?**
- Compare the three regimes on final test error and on how long each takes. In which of them do the elbows move, and why does that matter?**

Contributors:

- Anant Sahai.
- Sheng Shen.
- Gireeja Ranade.
- Matteo Guarrera.
- Mert Cemri.
- Sizhe Chen.
- Saagar Sanghavi.
- Kumar Krishna Agrawal.
- Kevin Li.